

EST-1141

Lenguajes de Programación

Juan Zamora O.

El lenguaje de programación JAVA



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

Estructura

1 Introducción

Características del Lenguaje

- Libera automáticamente la memoria utilizada mientras el programa corre
 - ***Garbage collection***
- Incluye una amplia variedad de librerías (bases de datos y diseño de interfaces de usuario entre otras)
- Ejecución independiente de la plataforma
 - Debido a su enfoque híbrido de compilación e interpretación
- Orientación a objetos clara: Datos y operaciones agrupadas dentro de clases

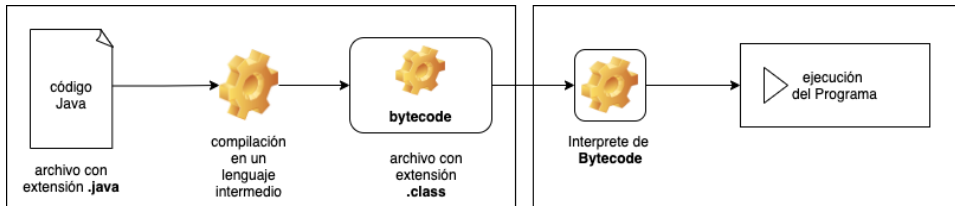
Bytecode

- Existe una gran diversidad de arquitecturas de máquina
- Se introduce como una solución transversal (***cross-platform***)
 - Se evita tener que desarrollar un compilador para cada tipo de máquina
- Permite combinar tecnología de compilación e interpretación

- Primero, cada código Java es compilado en un lenguaje intermedio denominado ***bytecode***
- Se detectan errores sin tener que ejecutar el programa
- bytecode **no** es lenguaje de máquina
- Un Interprete traduce de bytecode al lenguaje específico de la máquina en que se está ejecutando
- Bytecode es una especie de lenguaje de máquina
 - Pero de una máquina de ficción...**virtual**

Ventajas

- Al comparar con un lenguaje interpretado
 - Errores detectados en tiempo de compilación
- Al comparar con un lenguaje compilado
 - JVM permite portabilidad
 - código compilado puede ser ejecutado en cualquier plataforma
 - Por ejemplo, en una red de computadores con distintos sistemas operativos y/o procesadores



Este primer bloque es independiente de la plataforma en que se ejecutará el código

Este interprete está disponible para múltiples plataformas

Java Runtime Environment (JRE)

- Es el entorno de software en el cual se ejecutan programas escritos en el lenguaje Java
- Consiste de varios componentes
 - La API (Application Programming Interface)
 - Cargador de clases
 - Verificador de bytecode
 - La máquina virtual (JVM)

La API nativa

- Conjunto de rutinas pre-empaquetadas y listas para ser usadas
- Comunmente denominadas librerías
- Permite a las programadoras preocuparse específicamente de lo que necesitan resolver en lugar de tener que partir siempre desde 0
- Ejemplos: `javax.swing`, `java.io`, `java.math`

Cargador de clases

- Localiza, lee y carga en memoria los archivos con extensión `.class` generados anteriormente por el compilador bytecode
- En ocasiones las librerías están empaquetadas físicamente en archivos JAR (son como una especie de archivo **comprimido** ZIP)
- Siempre se carga el núcleo de librerías (jre/lib)

Verificador de Bytecode

- Componente que se asegura de la validez del código bytecode
- Verificación de tipos de variables y expresiones
- Se asegura de un uso seguro de la memoria
- Es posible des-habilitarlo para una ejecución más rápida
- Una vez validado un código, podrá ser ejecutado en la Máquina Virtual

La Máquina Virtual de Java (JVM)

- Especie de computador abstracto capaz de ejecutar únicamente bytecode en lugar de secuencias binarias
- Es el corazón de la filosofía “Escribe una vez, ejecuta donde sea”
- Existen varias implementaciones de JVM
 - La de Oracle es la más popular
- Una parte importante de la JVM es su interprete bytecode
- Otra es el **recolector de basura**
- Otra es el compilador **Just-in-time** que compila algunas instrucciones e interpreta otras

Estructura del Lenguaje

- Java no requiere de una indentación especial
- Los programas pueden ser formateados de cualquier forma
- Cada sentencia termina con ;
- Una sentencia realiza una acción específica y puede abarcar varias líneas

```
public class BMICalc {  
    // Declara variables  
    double peso;  
    double altura;  
    double BMI;  
  
    // metodo constructor  
    public BMICalc(double p, double a) {  
        peso = p;  
        altura = a;  
    }  
    // ... continua en lamina siguiente
```

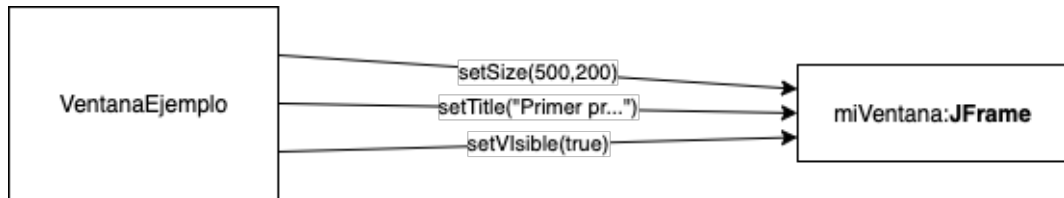
```
// continuación de código anterior
    public double calculaBMI() {
        return peso / (altura * altura);
    }

// Este es el punto de entrada para el inicio de
// la ejecución del programa.
public static void main(String[] args) {
    BMICalc calculadora = new BMICalc(60, 1.70);
    double bmi = calculadora.calculaBMI();
    // print BMI to screen
    System.out.println("Su BMI es " + bmi + ".");
}
}
```

```
import javax.swing.*;

class VentanaEjemplo{
    public static void main(String[] args) {
        JFrame miVentana;
        miVentana = new JFrame();
        miVentana.setSize(500, 200);
        miVentana.setTitle("Primer programa en "+
            "Java con ventanas");
        miVentana.setVisible(true);
        miVentana.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);
    }
}
```

El diagrama para el programa anterior es:



Clases

- En Java todo el código se agrupa en clases
- La definición comienza con un ***modificador de acceso***
 - Indica qué clases tienen acceso a ella (***a revisar más adelante***)
- Le sigue la palabra `class` y luego el nombre de la clase
- Todas las definiciones de clase están encerradas entre `{ }`
- El método `main` es especial
 - Es el punto de entrada para la ejecución del programa
 - Cuando la clase `BMICalc` es ejecutada por el JRE, comenzará por este método

Variables

- Toda definición de clase consiste de variables y métodos
- Una variable representa una ubicación de la memoria que almacena un valor específico
- Este valor puede cambiar durante la ejecución del programa
- Cada variable debe ser siempre definida indicando su tipo asociado

Por ejemplo: `double peso;`

Métodos

- Pieza de código dentro de una definición de clase
- Desempeña un tipo específico de funcionalidad
- Su definición siempre comienza con el modificador de acceso
- Continúa con la palabra **static** cuando **no** necesita de un objeto para ser invocado
- luego el tipo asociado con el retorno del método
- El identificador del método y sus parámetros

Clases y Objetos en JAVA

- EN el paradigma OO no existen constantes, variables o funciones aisladas
- Entonces no es posible definir tipos aislados, tienen que ser clases

Sin embargo, existen tipos primitivos que siguen existiendo (`int`, `double`, `boolean`, `byte`, `float`, `long`, `short`, `double` y `void`).

- Cada tipo primitivo es tratado como una instancia de una clase. Por ejemplo, `int` es visto como una instancia de la clase `Integer`.
- Esta conversión (***boxing***) y la conversión inversa (***unboxing***)
- Por lo tanto es totalmente válido el siguiente código:

```
Double d1 = 5.4;  
double d2 = new Double(3.3);
```

Ejemplos

- 1 Cree un archivo ***Estudiante.java*** con el siguiente contenido

```
class Estudiante{  
    int id;  
    String nombres;  
    String apellidoP;  
    String apellidoM;  
    int annoNac, mesNac, diaNac;  
}
```

② Cree un archivo ***Curso.java*** con el siguiente contenido

```
class Curso{  
    int id;  
    String nombre;  
}
```

③ Ahora construya el programa principal:

```
public class Ejemplo01 {  
    public static void main(String[] args){  
        Estudiante e01 = new Estudiante();  
        e01.id = 1099182;  
        e01.nombres = "Juan Antonio";  
        e01.apellidoM = "Labra";  
        e01.apellidoP = "Olivares";  
        Curso cur1 = new Curso();  
        cur1.id = 982;  
        cur1.nombre = "Lenguajes de programación";  
  
        System.out.println("Usted creó lo siguiente:");  
        System.out.println(e01.nombres+  
            ", " + e01.apellidoP+" "+e01.apellidoM);  
        System.out.println(cur1.id+" "+cur1.nombre);  
    }  
}
```


Ejercicios

- Construya un método que permita generar una representación de cada objeto como String
 - Nombre y apellido de cada estudiante
 - id y nombre de cada curso
- Construya una función que permita crear objetos Estudiante entregando directamente el nombre, apellido y fecha de nacimiento

Proble

